



CLASS 11 · WEEK 11 · FALL 2026

Intelligent Devices with AI

How a voice-driven “poetic light” came together with an ESP32, cloud APIs, and Claude — and what that build teaches about working with AI.

Thu, Nov 19

Case study: the poetic light

Human vs AI roles

No programming required



FINAL PROJECT CONCEPTS

Adding AI to a device **to do smart things**

No programming required — connect your device to cloud APIs and AI services, then describe what you want it to do.

Connect an ESP32 to cloud APIs

An API (application programming interface) is how your board asks another service to hand back data or do something.

EXAMPLE APIS

Weather

Music

AI chat

Over Wi-Fi, your board sends a request and gets an answer back — a forecast, a song, or a reply from a model. The same pattern works for almost any service you can find.



Find an API, get a key, **make requests**

01

Find an API

Pick the service that meets your needs — weather, music, AI chat, or something else entirely.

02

Register for a key

Sign up with the API to obtain an API key tied to your account.

03

Make requests

Use the API key to send requests — and get data or actions back.

The poetic light: **one sentence becomes a device**

THE ORIGINAL IDEA

"I want to say a word, and my poetic light will recite a related poem while changing to a color that represents it."

Built with an ESP32, cloud APIs, and Claude as the coding partner.



From idea to **working light**

The real build, in the order it happened.

01 Describe the idea

Start with one sentence: say a word, get a poem and a color.

02 Recording won't save

The voice recording can't be written to memory.

03 SD card struggles

Getting the SD card to work takes many attempts.

04 Firmware switch

New ESP32 firmware unlocks the full amount of RAM available.

05 Pulse red like HAL

Test and enhance: pulse red like HAL 9000.

06 Color switching works

The first feature lands — the light responds to a word.

07 Add a spoken poem

Feature enhancement: recite the poem out loud.

08 Fix the audio

The small speaker sounds poor, so a DFPlayer Mini MP3 module joins the build.

09 Ran out of context

The chat runs out of context before the build is finished.

Read the build **conversation**

The whole project was built with Claude as a coding partner. Both parts of the conversation are public.

01

Part 1

The conversation, start to finish.

<https://claude.ai/share/9bbc45ed-c6c8-4f0e-aac4-7bfd5f31903c>

02

Part 2

The conversation, continued.

<https://claude.ai/share/1a778197-0943-43d6-b83e-06c0389232d5>

The engineering mindset (by Claude)

Incremental development

Started simple, added complexity gradually.

Problem decomposition

Broke the system into manageable components.

Iterative debugging

Tested, identified issues, refined solutions.

Constraint recognition

Understood hardware and software limits, then adapted.

Design tradeoffs

Made informed decisions when multiple solutions existed.

Documentation review

Referenced existing code to learn hardware capabilities.

Resource management

Solved memory issues by upgrading firmware.

Feedback-driven development

Used test results to guide the next step.

Read the full analysis: <https://claude.ai/public/artifacts/a3f738cb-3b58-4cd5-9e85-6dd64541d943>

Who does what **in the build**

AREA	HUMAN · ENGINEER	AI · ASSISTANT
Problem framing	Defines the goal, hardware, and constraints	Translates intent into architecture and a code plan
Design & decisions	Chooses the trade-offs, simplifies the system	Suggests multiple implementation options
Diagnostics	Runs tests, identifies root causes	Generates diagnostic tools and fixes
Integration	Manages timing, pin mapping, and system flow	Produces working code and module connections
Validation	Tests and interprets the results	Refines code based on feedback
Optimization	Balances performance, cost, and usability	Automates tuning, surfaces data-driven insights
Documentation	Summarizes lessons and decisions	Formats results, diagrams, and reports

Where humans excel

Systems thinking & constraint recognition

- Detected pin-sharing conflicts that AI initially missed
- Caught logical errors AI couldn't infer from code alone

Diagnostic reasoning

- Found that low memory was a firmware issue, not a coding issue
- Designed and ran diagnostic tools to confirm issues empirically
- Validated AI suggestions against physical hardware behavior

Design judgment & simplification

- Eliminated SD card complexity once the firmware upgrade removed the constraint

Performance & tradeoff optimization

- Quantified "too slow" as "7 seconds" — a measurable metric
- Applied multi-criteria decision matrices (cost, latency, quality)

Engineering integrity

- Maintained pin mapping documentation and version control through iterations
- Directed the AI to restore baseline configurations when code drift occurred

Where AI excels

Rapid implementation

- Produced over 1,200 lines of functional MicroPython in a few hours
- Generated drivers, API calls, and WAV header logic quickly and accurately
- Integrated the Whisper, GPT, and TTS APIs seamlessly

Breadth of knowledge

- Provided hardware-specific setup for ESP32, I2S, SPI, and PWM
- Suggested valid code alternatives when one failed (buffer handling, threading)

Debugging assistance

- Offered step-by-step fixes for exceptions and syntax errors
- Generated diagnostic snippets for the SD card, microphone, and speaker

Documentation & teaching

- Explained underlying causes, like MicroPython subset differences
- Created diagrams, summaries, and performance tables automatically

Acceleration factor

Compressed a 24–35 hour traditional build into 4.5 hours total — roughly a 5–8× faster development cycle.

Best practices for human–AI collaboration

01

Be specific

Clear prompts yield relevant, hardware-aware results.

02

Show evidence

Share code, errors, or logs so AI can reason, not guess.

03

Iterate rapidly

Short prompt–test–refine loops beat long single prompts.

04

Validate everything

Run, test, and verify before integrating it into your system.

05

Keep context updated

Remind AI of constraints after each major change.

06

Use diagnostics early

Build quick test scripts to confirm hardware and firmware state.

07

Simplify continuously

Remove unnecessary layers once constraints are resolved.

08

Reflect & document

Capture what worked and why — AI learns context, you keep the insight.

BEFORE YOU GO

Remember

Submit a self-reflection before end of day — and bring what the poetic light taught you to your final project.

WHERE TO FIND IT

Self-reflection

go.tufts.edu/reflect

Office hours

go.tufts.edu/meet_tvdv

Course materials

tuftsmaker.github.io/ENT-164